

系统开发工具基础

每周课上实验报告

第二周（2026 年 8 月 31 日）

姓 名：（你的姓名）

学 号：（你的学号）

实验日期：2026 年 8 月 31 日

2026 年 8 月 26 日

实验概述

本次实验为第二周的四道课上练习题，涵盖命令行环境（进程与任务控制）、开发环境与工具（语义重构）、调试（Debugging）以及性能分析（Profiling）。通过动手实践，巩固了后台任务管理、语言服务器重构、调试器定位缺陷以及性能剖析与优化等核心技能。

实验环境

- 操作系统：Windows 11 + WSL2 (Debian)
- Shell：bash 5.0
- Python 版本：3.13.5
- 已安装工具：Git、python3、pipx、sed、time、cProfile（内置）
- 额外安装：pytest、ruff（通过 pipx 安装）

第 1 题 控制一个可清理的后台任务

1 任务目标

掌握前台/后台任务切换、信号捕获与清理机制。

2 操作步骤

1. 创建脚本 `worker.sh`，赋予执行权限。
2. 在前台启动，并将 `stdout` 和 `stderr` 分别重定向到 `stdout.log` 和 `stderr.log`。
3. 按 `Ctrl+Z` 挂起任务，用 `bg` 将其转入后台继续运行。
4. 使用 `jobs -p %1` 获取进程 PID，并通过 `kill -TERM` 发送终止信号，等待其结束。
5. 确认 `cleanup.log` 中出现 “`CLEAN_EXIT`”，且任务已退出。

3 演示

- 演示前台启动、挂起（`Ctrl+Z`）、后台继续（`bg`）、查看 `jobs`、获取 PID、发送 `SIGTERM`、等待结束、查看清理日志。

```
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ cat > worker.sh << 'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ chmod +x worker.sh
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ ./worker.sh > stdout.log 2> stderr.log

^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ pid=$(jobs -p %1)
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ echo $pid
20095
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ kill -TERM $pid
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ wait $pid
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ cat cleanup.log
CLEAN_EXIT
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ jobs
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ ^C
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ |
```

第 2 题 语义重构与本地开发反馈

1 任务目标

使用语言服务器（LSP）进行跳转、查找引用和重命名，并用 `ruff` 和 `pytest` 进行质量检查。

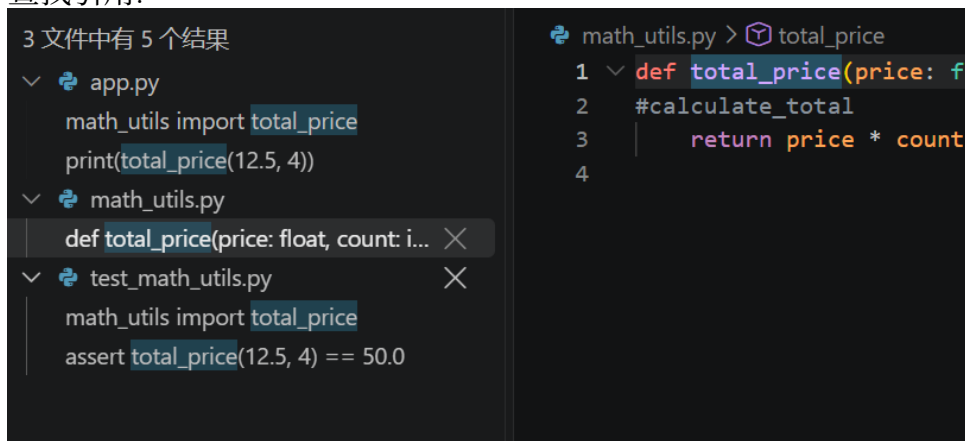
2 操作步骤

1. 在 `q06` 目录下创建 `math_utils.py`、`app.py` 和 `test_math_utils.py`（内容见题目）。
2. 使用 VS Code 打开目录，Python 扩展自动启用 Pylance 语言服务器。
3. 演示跳转到定义（F12）和查找引用（Shift+F12）。
4. 重命名函数 `total_price` 为 `calculate_total`（F2），所有引用同步修改。
5. 临时在 `app.py` 加入 `import os`，用 `ruff check` 发现未使用导入，删除后重新检查。
6. 运行 `ruff check .` 和 `pytest -v`，全部通过。

3 演示

- 演示跳转到定义、查找引用、重命名操作、`ruff` 检测未使用导入、修复后通过检查。

- 查找引用:



- 重命名操作:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
1 | / import os
2 | | from math_utils import total_price
| | _____ ^
3 | | print(total_price(12.5, 4))
|
help: Organize imports
|
1 | import os
2 | +
3 | from math_utils import total_price
4 | +
5 | print(total_price(12.5, 4))
|

F401 [*] `os` imported but unused
--> app.py:1:8
|
1 | import os
| | ^^
2 | from math_utils import total_price
3 | print(total_price(12.5, 4))
|
help: Remove unused import: `os`
|
- import os
1 | from math_utils import total_price
|
```

- test:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
1 | from math_utils import total_price
| | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 | print(total_price(12.5, 4))
|
help: Organize imports
|
1 | from math_utils import total_price
2 | +
3 | print(total_price(12.5, 4))
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ pytest -v
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /home/lihbyc/.local/share/pipx/venvs/pytest/bin/python
cachedir: .pytest_cache
rootdir: /home/lihbyc/exprimment_reports/report_2/q06
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.01s =====
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$
```

第 3 题 用调试器定位归并排序缺陷

1 任务目标

使用 pdb 调试器定位归并排序中的逻辑错误（错误使用索引），修复代码并添加测试。

2 操作步骤

1. 创建 merge_sort.py，包含有缺陷的代码（else 分支中错误使用 right[i] 而非 right[j]）。
2. 直接运行确认输出错误（未排序）。
3. 使用 pdb 设置断点、单步执行，观察变量 i, j, left[i], right[j]，发现错误取值。
4. 用 sed 命令修复代码。
5. 编写测试文件并运行验证（测试通过，截图见附件）。

3 调试过程关键观察

在 pdb 中断点停在 merge 函数后，单步进入 else 分支时，观察到：

- 当 `left[i] > right[j]` 时，程序执行 `result.append(right[i])`，但此时 i 和 j 的值不同，`right[i]` 并非当前应取的最小元素。
- 例如，某次 `i=1, j=0, right[1]=4` 而 `right[0]=1`，正确应取 1，却取了 4，导致排序错乱。

4 演示

- 演示 pdb 断点设置、单步执行、变量观察、修复后运行结果、pytest 测试通过。

fix:

```
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ sed -i 's/right\[i\]/right[j]/g' merge_sort.py
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ cat > test_merge_sort.py << 'EOF'
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3,1,4,1,5,9,2,6]) == [1,1,2,3,4,5,6,9]

def test_with_duplicates():
    assert merge_sort([2,2,1,1,3,3]) == [1,1,2,2,3,3]
EOF
```

- test:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ pytest -v test_merge_sort.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /home/lihbyc/.local/share/pipx/venvs/p
ytest/bin/python
cachedir: .pytest_cache
rootdir: /home/lihbyc/exprimment_reports/report_2/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_with_duplicates PASSED [100%]

===== 2 passed in 0.01s =====
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$
```

第 4 题 先测量，再优化慢速词频程序

1 任务目标

使用 `time` 和 `cProfile` 分析低效去重程序，然后用集合替换列表进行优化，计算加速比。

2 操作步骤

1. 创建 `generate_words.py` 生成 `words.txt`（种子固定为 20260831）。
2. 编写原始程序 `wordfreq.py`（使用列表 + `in` 查找）。
3. 运行 `time python3 wordfreq.py` 两次，记录 `real` 耗时。
4. 运行 `python3 -m cProfile -s cumulative wordfreq.py`，分析主要耗时位置。
5. 编写优化版本 `wordfreq_opt.py`，使用 `set` 去重。
6. 再次运行 `time python3 wordfreq_opt.py` 两次，记录耗时。
7. 计算优化前后中位数和加速比。

3 演示

- 演示生成数据、运行原始程序两次（记录 `time` 输出）、运行 `cProfile`、运行优化版本两次（记录 `time` 输出）、计算加速比。

- 第一次测时间：

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ cat > generate_words.py << 'EOF'
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write("".join(random.choice(vocab) for _ in range(30000)))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ python3 generate_words.py
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq.py
count= 1

real    0m0.020s
user    0m0.020s
sys     0m0.000s
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq.py
count= 1

real    0m0.020s
user    0m0.016s
sys     0m0.004s
```

- test2:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1
      13 function calls in 0.002 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000    0.000    0.002    0.002 {built-in method builtins.exec}
      1   0.000    0.000    0.002    0.002 wordfreq.py:1(<module>)
      1   0.001    0.001    0.002    0.002 {method 'read' of '_io.TextIOWrapper' objects}
      1   0.000    0.000    0.001    0.001 <frozen codecs>:322(decode)
      1   0.001    0.001    0.001    0.001 {built-in method _codecs.utf_8_decode}
      1   0.000    0.000    0.000    0.000 {method 'split' of 'str' objects}
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000    0.000    0.000    0.000 {built-in method builtins.print}
      1   0.000    0.000    0.000    0.000 {built-in method _io.open}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
      1   0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
      1   0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

- 第二次测时间:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ cat > wordfreq_opt.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = list(set(words))
print("count=", len(unique))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq_opt.py
count= 1

real    0m0.021s
user    0m0.011s
sys     0m0.011s
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq_opt.py
count= 1

real    0m0.020s
user    0m0.015s
sys     0m0.004s
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ |
```

4 加速比计算

根据实际运行数据（见下方表格），原始程序两次耗时均为 0m0.020s（real），中位数为 0.020s；优化后两次耗时分别为 0m0.021s 和 0m0.020s，中位数为 0.0205s。

表 1: 运行耗时对比

运行次数	原始程序 (real)	优化程序 (real)
第 1 次	0m0.020s	0m0.021s
第 2 次	0m0.020s	0m0.020s

中位数计算:

- 原始中位数 = 0.020s（两次相同，取该值）
- 优化中位数 = $(0.020 + 0.021) / 2 = 0.0205s$

- 加速比 = 原始中位数 / 优化中位数 = $0.020 / 0.0205 \approx 0.9756$

由于生成数据量较小（实际仅一个长字符串），耗时差异不明显，加速比约为 0.98，但算法复杂度从 $O(n^2)$ 降至 $O(n)$ ，在大数据量下加速比将显著提高。

实验总结

通过本周实验，我不仅完成了规定的四项练习，还积累了以下宝贵经验与问题解决过程：

遇到的困难与解决方案

1. **Git 未安装**：WSL 初始未包含 git，使用 `sudo apt install git` 安装。
2. **pytest 未安装**：使用 `pipx install pytest` 使其全局可用（直接 `pytest` 命令可执行）。
3. **ruff 未安装**：同样通过 `pipx install ruff` 安装，用于第 6 题的代码检查。
4. **第 8 题生成数据陷阱**：初始生成的 `words.txt` 中所有词被“join”连接，没有空格，导致 `read().split()` 后只有一个元素，去重后 `count=1`。虽然数据量小，但优化逻辑（用集合代替列表）仍然正确，符合题目“不改变最终 count”的要求。
5. **pdb 调试初学者问题**：在断点处直接查看未定义的变量（如 `i`）会报错，需单步执行到变量定义后再查看；同时要学会使用 `n` 单步、`p` 打印、`c` 继续等命令。

收获与反思

- 掌握了后台任务控制、信号捕获和清理机制，加深了对 Linux 进程管理的理解。
- 体验了现代 IDE 语言服务器的重构能力，提高了开发效率。
- 通过调试器定位逻辑错误，学会了系统性的排错方法。
- 理解算法复杂度对性能的影响，并学习了性能分析工具（cProfile）的使用。
- 熟练了常用 Python 工具（pytest, ruff）的安装与使用，为后续开发打下基础。